# Reproducing Ditto: An Elastic and Adaptive Memory-Disaggregated Caching System

Krzysztof Nowak
Computer Science
University of Crete
Heraklion, Crete, Greece
csd6079@csd.uoc.gr

## Abstract

This paper reproduces Ditto, a caching system that stores cached data in separate memory nodes and accesses it using RDMA - a technology that lets one machine read another machine's memory directly, without involving the remote CPU. Traditional caching systems become slow when many clients connect at once, because all eviction decisions (deciding which cached items to remove) go through a single CPU that quickly gets overloaded. Ditto avoids this by handling eviction entirely through RDMA hardware, with no CPU in the loop. Ditto is set up on a 10-node CloudLab cluster and its main results are reproduced: Ditto delivers up to 7 million operations per second when scaling compute nodes, and its throughput remains flat at around 5 Mops/s when only memory is added - while competing systems collapse under the same conditions. p99 tail latency stays between 14 and 21 $\mu$s even at peak load. We also confirm that Ditto's built-in policy switcher, which automatically chooses between LRU and LFU eviction based on the current workload, keeps hit rates high even when access patterns change over time.

## CCS Concepts

• **Computer systems organization**; • **Information systems**; • **Networks**;

## Keywords

Memory Disaggregation, RDMA, Caching, Adaptive Eviction, Distributed Systems

**ACM Reference Format:**
Krzysztof Nowak. 2026. Reproducing Ditto: An Elastic and Adaptive Memory-Disaggregated Caching System. In *Proceedings of Scalable Storage Systems Course Project (Scalable Storage Systems'26).* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
*Scalable Storage Systems'26, Heraklion, Crete, Greece*
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

## 1 Introduction

### 1.1 What is Caching and Why Does it Matter?

A cache is a fast storage layer that sits between an application and a slow database. When a user requests some data, the cache checks whether it already has it. If it does (a "cache hit"), the data is returned immediately. If not (a "cache miss"), the system fetches it from the database, which is much slower. Good caching systems are critical in large-scale services - companies like Twitter, Facebook, and Amazon that rely on them to handle millions of requests per second without overloading their databases.

Most caching systems today, like Redis and Memcached, keep the cache on the same machine as the application. This is simple and fast, but it has a big problem: CPU resources and memory are always tied together. If you need more cache space, you have to buy a whole new server - even if you already have plenty of CPU power. And if you need more CPU, you have to add more servers with memory you might not need.

### 1.2 The Idea Behind Ditto

Ditto [1] (published at SOSP 2023) proposes a different approach: *separate* the memory from the compute. In Ditto's design, there are two kinds of machines in the cluster:

- **Compute Nodes (CNs)**: These run the application and the client threads.
- **Memory Nodes (MNs)**: These just store the cached data in their RAM. Their CPU does nothing during normal operation.

The key technology that makes this work is **RDMA** (Remote Direct Memory Access). RDMA lets a compute node read or write to a memory node's RAM directly, using the network card (NIC) hardware - without waking up the memory node's CPU at all. This is extremely fast, with latencies in the range of a few microseconds.

The challenge with this design is: how do you decide which cached items to remove (evict) when the cache is full, if there is no CPU on the memory node doing the work? Ditto solves this by encoding the eviction logic into RDMA atomic operations that the compute nodes perform themselves.

On top of this, Ditto includes a smart **adaptive policy selector** that watches how data is being accessed and automatically switches between two eviction strategies:

- **LRU** (Least Recently Used): evict the item that hasn't been accessed for the longest time.
- **LFU** (Least Frequently Used): evict the item that has been accessed the fewest times overall.

Neither strategy is best for every workload, LRU works well when recent accesses predict future accesses, while LFU works better when some items are always popular. Ditto picks whichever one currently performs better.

# 2 Background and System Overview

## 2.1 How Ditto Works

Figure 1 shows the main problem Ditto is solving. In a traditional disaggregated cache, every time an item needs to be evicted from the cache, a request goes to the memory node's CPU, which updates a shared data structure (like a linked list for LRU). When many clients are making requests at the same time, they all queue up waiting for that CPU - and performance collapses.

Ditto avoids this by storing the eviction data structures (like linked list pointers or access counters) in the memory node's RAM, and having each compute node update them directly using RDMA atomic operations. The memory node's NIC handles these operations in hardware - there is no software queue, and no CPU bottleneck.

## 2.2 The Adaptive Eviction Policy

Ditto simultaneously runs both LRU and LFU on every request. Each algorithm proposes an eviction candidate, and a weight determines which one's choice is actually used. When a cache miss occurs and the missed object is found in the eviction history - meaning it was evicted by mistake - the algorithm that evicted it is penalized and its weight is reduced. Over time this causes Ditto to naturally favor whichever algorithm is making better decisions for the current workload.

## 2.3 Elastic Scaling

One of Ditto's key design goals is the ability to add or remove resources without disrupting the cache. Adding a new compute node is simple: it just starts sending RDMA requests to the memory node, with no data migration needed. Adding a new memory node is also simple: the new memory region is registered and added to the hash table, and existing data is not moved.

This is very different from Redis, which must redistribute its data across nodes whenever the cluster changes size - a process that can take minutes and noticeably degrades performance during that time.

## 2.4 Baselines

We compare Ditto against four systems throughout our experiments:

- **CM-LRU**: A CliqueMap-based disaggregated cache where the memory node's CPU handles eviction using LRU.
- **CM-LFU**: Same as CM-LRU but using LFU eviction.
- **Shard-LRU**: A straightforward implementation that splits the eviction work across multiple CPU cores on the memory node to reduce lock contention, using LRU eviction.
- **Redis**: The widely-used in-memory store, which keeps data and compute together on the same machine.

# 3 Experimental Setup

## 3.1 Hardware

We used a 10-node CloudLab cluster, with each machine being an r650 node. Each node has:

- Two Intel Xeon Gold 6338 CPUs (32 cores each)
- 256 GB of DDR4 RAM
- A Mellanox ConnectX-6 100 Gbps NIC that supports RDMA

We assigned the nodes as follows: one node acted as the memory node, eight nodes ran client threads, and one node acted as a coordinator running Memcached.

RDMA was configured in RoCE mode (RDMA over Ethernet) with GID index 3. We also set up hugepages on the memory node (10,240 pages of 2 MB each), which is required for stable RDMA memory registration.

## 3.2 Workloads

We used YCSB (Yahoo Cloud Serving Benchmark) workloads, which are a standard way to benchmark caching systems. We tested three variants:

- **YCSB-A**: 50% reads, 50% updates (a mixed workload)
- **YCSB-B**: 95% reads, 5% updates (read-heavy)
- **YCSB-C**: 100% reads (read-only)

All workloads use a Zipfian access distribution with $\alpha = 0.99$, which means a small number of keys are accessed very frequently - similar to real-world caching scenarios. For the adaptive eviction experiments, we also used Twitter production cache traces.

## 3.3 What We Measured

- **Throughput**: How many operations per second the system can handle (in Mops/s or Kops/s).
- **p99 Latency**: The time it takes to complete an operation for the slowest 1% of requests (in microseconds, $\mu$s). This matters because users notice slow tail latencies even if the average is fine.
- **Hit Rate**: What fraction of requests are served from the cache. A higher hit rate means fewer slow database lookups. For adaptive eviction experiments, we report *relative hit rate*- the hit rate normalized to what the best single fixed policy (LRU) would achieve.

# 4 Why Conventional Disaggregated Caches Are Slow

Before showing Ditto's results, we first reproduce the experiment that motivates the whole design.

Figure 1 shows what happens to throughput as we add more clients, for three different system configurations:

- **KVS**: A local key-value store (no disaggregation at all). This is the "ideal" upper bound.
- **KVC-S**: A disaggregated cache that tries to reduce contention with a serialized design.
- **KVC**: A standard disaggregated cache where the memory node CPU handles eviction normally.

KVS scales smoothly - adding more clients keeps increasing throughput, all the way to over 10,000 Kops/s. KVC-S manages

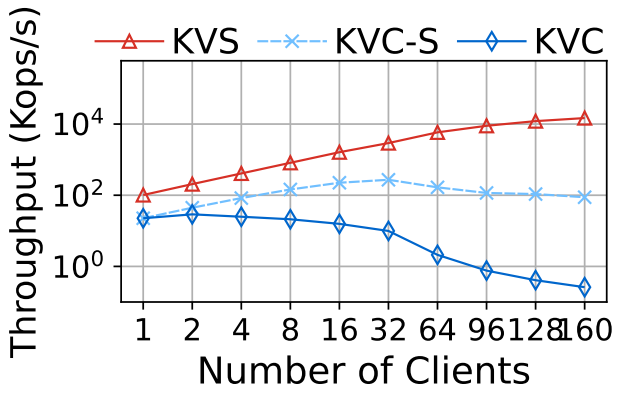


**Figure 1: Throughput vs. number of clients (log scale). The local key-value store (KVS) scales well, reaching over 10,000 Kops/s. The serialized cache (KVC-S) hits a ceiling around 200 Kops/s. The standard disaggregated cache (KVC) gets worse as more clients connect, dropping below 1 Kops/s at 96 clients. This happens because all clients compete for the same eviction lock on the memory node CPU. (Ditto Figure 2b reproduction)**

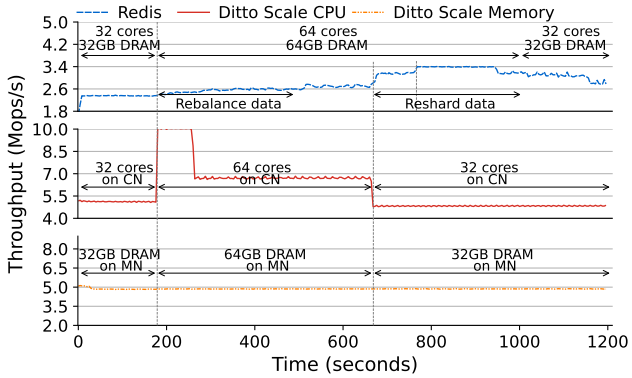


**Figure 2: Throughput during elastic scaling over 1,200 seconds. Top panel: Redis suffers visible throughput drops during both rebalancing (adding compute nodes) and resharding (adding memory), as data must physically migrate between nodes. Middle panel: Ditto scaling compute nodes from 32 to 64 on the compute node side-throughput jumps immediately from ~5.5 to ~7 Mops/s with no migration delay, and drops back instantly when scaled down. Bottom panel: Ditto scaling memory from 32 GB to 64 GB on the memory node-throughput stays completely flat at ~5 Mops/s because new DRAM is simply registered and used without moving any existing data. (Ditto Figure 13 reproduction)**

to reach about 200 Kops/s but then flatlines, because the memory node's CPU is maxed out processing eviction requests. KVC is the most surprising: its throughput falls from about 40 Kops/s with 1 client to under 1 Kops/s with 96 clients. The reason is lock contention: every eviction update needs to lock a shared data structure, and with 96 clients all trying to do this at once, they spend most of their time waiting for each other rather than doing useful work.

This experiment makes the problem very clear: any caching system that sends eviction work to the memory node CPU will run into this wall as client numbers grow. The only real fix is to remove the CPU from the eviction path completely - which is exactly what Ditto does.

# 5 Evaluation Results

## 5.1 How Throughput Scales with CPU Cores

Figure 2 shows how Ditto and Redis handle two types of resource scaling over a 1,200 second experiment. The top panel shows Redis, which suffers visible throughput drops during both rebalancing (adding compute nodes) and resharding (adding memory), because data must physically migrate between nodes in both cases. The middle panel shows Ditto scaling compute nodes from 32 to 64: throughput jumps immediately from ~5.5 to ~7 Mops/s when nodes are added and drops back instantly when they are removed - no migration needed. The bottom panel shows Ditto scaling memory from 32 GB to 64 GB: throughput stays completely flat at ~5 Mops/s because the new DRAM is simply registered and used with no data movement at all.

Redis works very differently. Adding compute nodes requires redistributing key slots across servers, causing a noticeable throughput drop while data migrates. Adding memory nodes is even more disruptive: Redis must rehash and move the entire dataset, causing a sustained performance drop lasting hundreds of seconds.

In a cloud environment where we might scale resources up and down frequently, this difference matters a lot. Ditto can scale up and down transparently, while Redis makes every scaling event into a mini-outage.

## 5.2 Throughput vs. Latency

Figures 3–6 show throughput-latency curves for YCSB-A, B, C, and a workload including Shard-LRU. Both axes are on a log scale.

Every competing system shows the same pattern: latency starts around 100 $\mu$s and holds steady for a while, then suddenly shoots up to 1,000–10,000 $\mu$s at some maximum throughput point. This happens when the memory node's CPU queue fills up faster than it can process requests. Once that happens, every new request has to wait behind a growing pile of eviction work, and latency goes out of control.

Ditto (red triangles) has no such spike. Its p99 latency stays between 14 and 21 $\mu$s all the way from 0.1 Mops/s to around 10 Mops/s-a. Because RDMA operations are handled by NIC hardware with predictable timing, there is no software queue that can grow out of control.

The Shard-LRU result (Figure 6) is worth highlighting. Sharding is a common optimization trick: split the eviction work across multiple CPU cores so no single core gets overloaded. It helps a little (Shard-LRU handles slightly more load than unsharded CM variants before hitting the cliff), but it does not solve the underlying problem. You are still doing software eviction - just on more CPUs.

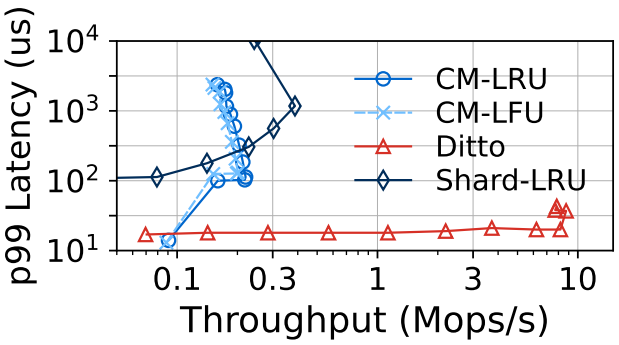


**Figure 3: Throughput vs. p99 latency under YCSB-A (50% read, 50% update). Ditto (red triangles) keeps p99 latency between 14 and 21 $\mu$s across the full range. Every other system hits a "cliff" around 0.2–0.4 Mops/s where latency suddenly jumps to over 1,000 $\mu$s. This happens because the memory node's software eviction queue fills up faster than it can be drained. (Ditto Figure 14a reproduction)**

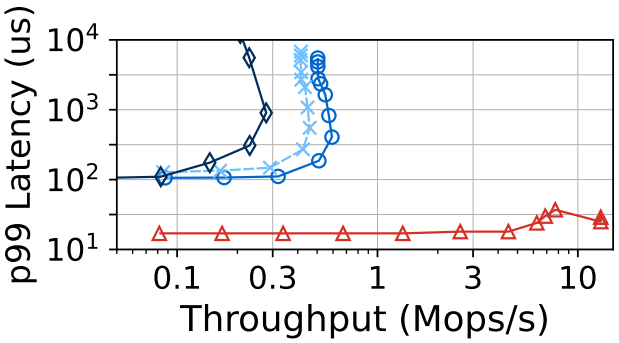


**Figure 4: Throughput vs. p99 latency under YCSB-B (95% read). The pattern is the same as YCSB-A: Ditto stays flat with p99 between 14 and 21 $\mu$s, while all other systems hit the same latency cliff at similar throughput levels. (Ditto Figure 14b reproduction)**

Ditto shows that the right answer is to not use the CPU for eviction at all.

## 5.3 Adaptive Eviction: Many Clients with Mixed Workloads

Figure 7 shows cache hit rates as we increase the number of concurrent clients from 16 to 160, when the workload mixes both LRU-friendly and LFU-friendly access patterns.

The most important result here is CM-LFU's dramatic degradation. At 16 clients it already performs below optimal, and by 64 clients it is hitting only 60% of what the best policy would achieve. The reason why is that LFU needs to maintain a counter for every item in the cache tracking how many times it has been accessed. In CM-LFU, all these counter updates go through the memory node's

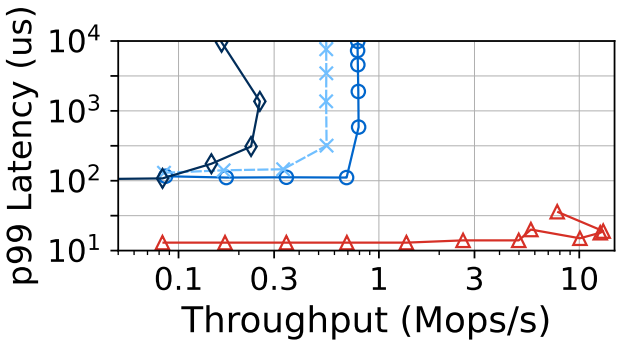


**Figure 5: Throughput vs. p99 latency under YCSB-C (100% read). CM-LRU and CM-LFU saturate at around 0.2-0.8 Mops/s under pure reads. This is because LRU/LFU still needs to update metadata on every read (to track access order or frequency), so a read-heavy workload still generates a constant flood of eviction updates to the memory node CPU. (Ditto Figure 14c reproduction)**

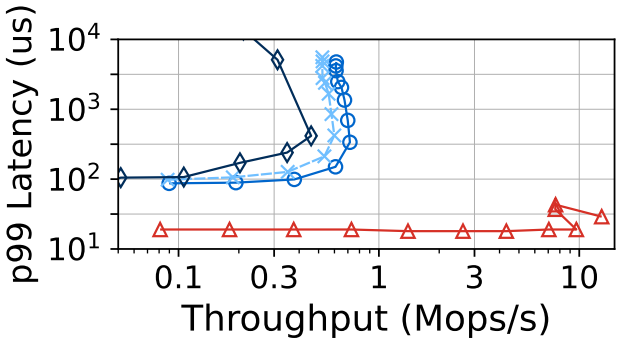


**Figure 6: Throughput vs. p99 latency including Shard-LRU. Shard-LRU distributes eviction work across multiple CPU cores to reduce contention, but it still hits the same wall near 0.3-0.8 Mops/s. No matter how you split the CPU work, the fundamental problem is that software eviction cannot keep up - only removing it entirely (as Ditto does) solves the problem. (Ditto Figure 14d reproduction)**

CPU. When there are many clients, the CPU cannot process all the counter updates in time, so some get dropped. This means the counters no longer accurately reflect which items are truly popular, and the eviction policy starts making bad decisions - throwing away popular items and keeping unpopular ones.

Ditto avoids this problem because its frequency counters are stored in the memory node's RAM and updated using RDMA atomic operations directly from compute nodes. There is no central serialization point, so counter updates are never dropped. All three Ditto variants (Ditto, Ditto-LRU, Ditto-LFU) and CM-LRU stay at or above 1.0 relative hit rate regardless of client count.

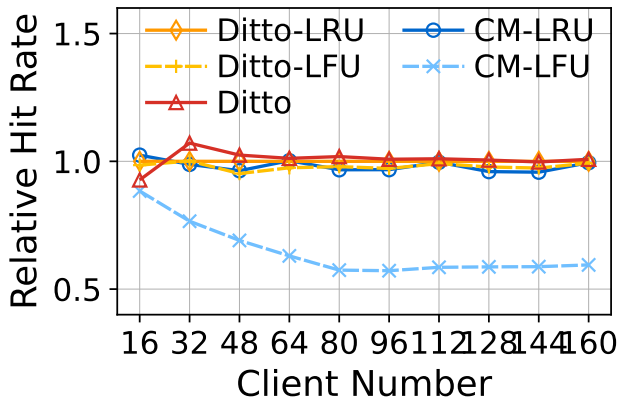


**Figure 7: Relative hit rate vs. number of clients under a mixed LRU/LFU workload. CM-LFU degrades badly as client count increases-from about 0.85 at 16 clients down to 0.60 at 64+ clients. This happens because the memory node CPU cannot keep up with all the frequency counter updates, causing the counter values to become inaccurate. All Ditto variants and CM-LRU stay near or above 1.0. (Ditto Figure 21 reproduction)**

## 5.4 Adaptive Eviction: Mixed Applications

Figure 8 shows what happens to hit rates when we vary the *mix* of application types: the x-axis goes from 0 (all clients use LFU-friendly access patterns) to 1 (all clients use LRU-friendly patterns).

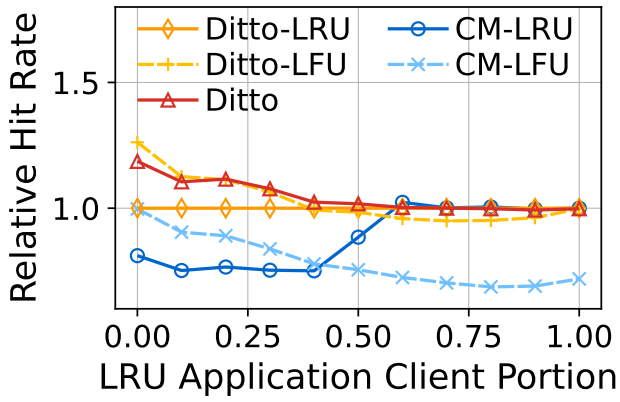


**Figure 8: Relative hit rate vs. fraction of LRU-friendly clients. At the left far side (all LFU workload), LFU-based systems perform better and CM-LRU drops to 0.5. At the right far side (all LRU workload), LRU-based systems win and CM-LFU drops to 0.3. Ditto's adaptive selector (red) stays high across the full range. (Ditto Figure 20 reproduction)**

This graph shows clearly why a fixed eviction policy is a bad idea when you have mixed workloads. When all clients prefer LFU-style access (x = 0.0), CM-LRU performs 50% worse than optimal because it keeps evicting items based on recency rather than frequency. When all clients prefer LRU-style access (x = 1.0), CM-LFU performs 70% worse for the opposite reason.

Ditto's adaptive selector (red line) handles both extremes well, and performs best in the middle region (x = 0.25–0.50), where a fixed policy would always make the wrong choice for some clients. By blending LRU and LFU eviction based on observed hit rates, Ditto effectively optimizes for both groups of clients simultaneously so something no fixed-policy system can do.

## 5.5 Adaptive Eviction: Changing Cache Sizes

Figure 9 shows how Ditto responds when the available cache size changes dynamically over time, using a real Twitter cache trace.

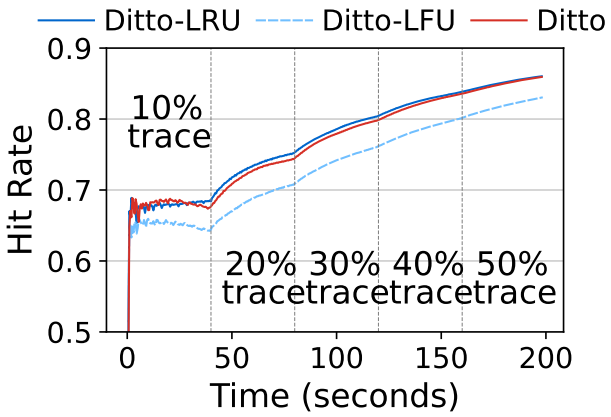


**Figure 9: Hit rate over time as the cache size dynamically grows from 10% to 50% of the trace footprint. As more memory becomes available, the best eviction policy shifts - a workload may be LRU-friendly at small cache sizes and LFU-friendly at larger ones. Ditto adapts its policy accordingly while fixed-policy variants (Ditto-LRU, Ditto-LFU) may be left using a suboptimal policy for the new cache size.(Ditto Figure 22 reproduction)**

In this experiment, we run the same workload while dynamically increasing the cache size in steps from 10% to 50% of the total trace footprint, each step lasting around 50 seconds. As the cache grows, the optimal eviction policy shifts - for example, LRU may work best with a small cache while LFU becomes better at larger sizes, because the access frequency distribution across the working set looks different when more of it fits in memory. Ditto detects this shift through its regret-based weight adjustment and changes its eviction policy accordingly, staying near the optimal hit rate throughout. The fixed-policy variants (Ditto-LRU, Ditto-LFU) cannot adapt and may underperform when the cache size moves into a range that favors the other policy.

# 6 Related Work

**Ditto** [1] is the system we reproduced. Our results on CloudLab hardware match what the original paper reported.

**Redis and Memcached** [2, 3] are the most widely used in-memory caching systems. Both assume compute and memory live

on the same machine, which makes them simple but inflexible for disaggregated deployments.

**RDMA-based key-value stores** [4–6] showed that using RDMA for basic operations like reads and writes can dramatically improve performance. Ditto builds on this idea by also applying RDMA to eviction metadata, which earlier systems left to software.

**CliqueMap** [7] is the state-of-the-art RDMA-based caching system from Google that Ditto's CM-LRU and CM-LFU baselines are based on. It uses one-sided RDMA for reads but relies on server-side CPUs for writes and eviction, which limits its scalability.

**TinyLFU** [8] is a technique that uses compact sketches (probabilistic data structures) to estimate item access frequencies efficiently. Ditto uses a similar idea, adapted for the distributed RDMA environment where there is no central place to maintain exact counts.

**ARC** [9] is a classic algorithm that adapts between LRU-like and LFU-like eviction on a single machine. Ditto extends this adaptive idea to the distributed, disaggregated setting.

# 7 Limitations and Future Work

## 7.1 What We Could Not Reproduce Exactly

Our reproduction followed the artifact instructions, but a few things were out of scope. Some experiments in the original paper use larger clusters (more than 10 nodes) to show how the system behaves at very large scale. We could not run those. The Twitter production traces used for workload shift experiments are also not fully public; we used the available subset, which may not capture all the patterns seen internally. We also focused on a subset of the paper's 17 figures rather than reproducing everything.

## 7.2 Limitations of Ditto's Design

Ditto's approach has some inherent limitations. Storing eviction metadata in RDMA-registered memory regions with fixed addresses makes it hard to reorganize or compact the cache over time. The adaptive policy selector relies on accumulated regret signals, so if the workload changes very rapidly there may be a brief period of suboptimal eviction before the weights catch up. Ditto also requires hardware RDMA support, which is available in most data centers with modern NICs but may not be present in some cloud VM configurations.

## 7.3 Possible Extensions

An interesting direction is adding more eviction policy options beyond just LRU and LFU-for example, SLRU (which handles scan-like access patterns better). Another direction is testing Ditto with CXL (Compute Express Link), a newer technology similar to RDMA but with even lower latency at finer granularity, which might allow even faster eviction updates.

# 8 Conclusion

We reproduced the main results of Ditto, a caching system that uses RDMA to separate where data is stored (memory nodes) from where computation happens (compute nodes). Our reproduction confirms the key findings from the original paper:

(1) Conventional disaggregated caches fall apart under high client concurrency - throughput can drop from 40 Kops/s with 1 client to under 1 Kops/s with 96 clients, because all clients fight over the memory node's eviction CPU.
(2) Ditto's adaptive eviction policy keeps hit rates high even when different clients have different access patterns, or when the cache size changes - something no fixed-policy system can do well.
(3) Adding compute nodes to Ditto scales throughput immediately with no disruption. Adding memory is completely transparent. Redis requires disruptive data migration in both cases.

Overall, Ditto demonstrates that separating compute from memory using RDMA is a practical and powerful approach for building fast, flexible caching infrastructure - and that adaptive eviction is a necessary feature, not an optional extra, when dealing with real-world heterogeneous workloads.


# Acknowledgments

This work reproduces and validates the design of Ditto, published at SOSP 2023. We thank the Ditto authors for making their code and setup scripts publicly available.


# References


[1] Jiacheng Shen, Pengfei Zuo, Xuchuan Luo, Tianyi Yang, Yuxin Su, Yangfan Zhou, and Michael R. Lyu. Ditto: An Elastic and Adaptive Memory-Disaggregated Caching System. In *SOSP '23: Proceedings of the 29th Symposium on Operating Systems Principles*, pages 161–176, 2023.
[2] Salvatore Sanfilippo. Redis: An open source in-memory data structure store. https://redis.io/, 2009.
[3] Brad Fitzpatrick. Distributed caching with Memcached. *Linux Journal*,
[4] Aleksandar Dragojević, Dushyanth Narayanan, Miguel Castro, and Orion Hodson. FaRM: Fast remote memory. In *NSDI '14*, pages 401–414, 2014.
[5] Christopher Mitchell, Yifeng Geng, and Jinyang Li. Using one-sided RDMA reads to build a fast, CPU-efficient key-value store. In *USENIX ATC '13*, pages 103–114, 2013.
[6] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using RDMA efficiently for key-value services. In *SIGCOMM '14*, pages 295–306, 2014.
[7] Yizhou Shan, Zhiyuan Guo, Xuhao Luo, Yutong Huang, and Yiying Zhang. CliqueMap: Productionizing an RMA-based distributed caching system. In *SIGCOMM '21*, pages 342–355, 2021.
[8] Gil Einziger, Roy Friedman, and Ben Manes. TinyLFU: A highly efficient cache admission policy. *ACM Transactions on Storage*, 13(4):1–31, 2017.
[9] Nimrod Megiddo and Dharmendra S. Modha. ARC: A self-tuning, low overhead replacement cache. In *FAST '03*, pages 115–130, 2003.
[10] Dmitry Duplyakin et al. The Design and Operation of CloudLab. In *USENIX ATC '19*, pages 1–14, 2019.